

LAPORAN PRAKTIKUM PEKAN 9

STRUKTUR DATA

Disusun Oleh:

Muhammad Fharel

2511531010

Dosen Pengampu:

Dr. Wahyudi S.T.M.T

Asisten Pratikum:

Muhammad Galid Avero



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Segala puji penulis panjatkan ke hadirat Allah SWT atas rahmat dan karunia-Nya sehingga laporan praktikum Struktur Data pada tanggal 9 Juni 2026 yang membahas pembuatan struktur pohon biner (Binary Tree) yang terdiri dari beberapa simpul yang saling terhubung. Selain itu, dilakukan juga implementasi struktur data Graph dengan menggunakan algoritma penelusuran Breadth First Search (BFS) dan Depth First Search (DFS). Melalui praktikum ini mahasiswa dapat memahami bagaimana data disimpan dalam bentuk hierarki maupun dalam bentuk jaringan hubungan antar simpul.

Ucapan terima kasih ditujukan kepada dosen pengampu, asisten praktikum, serta rekan-rekan yang telah membantu dalam proses pelaksanaan praktikum. Penulis menyadari bahwa penulisan laporan masih jauh dari kata sempurna, oleh karena itu kritik dan saran sangat diharapkan untuk penyempurnaan di kemudian hari. Semoga laporan ini memberikan manfaat dan menambah wawasan pembaca.

Padang, 11 Juni 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan Praktikum	1
1.3 Manfaat Praktikum	2
BAB II PEMBAHASAN	3
2.1 Binary Tree dan Graph Traversal	3
2.2 Kode Pemograman	4
2.2.1 Class Node_2511531010	4
2.2.2 Class BTree_2511531010	5
2.2.3 Class BtreeDriver_2511531010.....	7
2.2.4 Output BtreeDriver_2511531010.....	9
2.2.5 Class GraphTraversal_2511531010	9
2.2.6 Output GraphTraversal_2511531010.....	12
BAB III PENUTUP	13
3.1 Kesimpulan.....	13
DAFTAR PUSTAKA	14
TUGAS PRAKTIKUM.....	15

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan salah satu konsep penting dalam pemrograman yang digunakan untuk mengatur, menyimpan, dan mengelola data agar dapat diproses secara efisien. Dalam pengembangan perangkat lunak, pemilihan struktur data yang tepat dapat meningkatkan performa program serta mempermudah proses pencarian dan pengolahan data.

Salah satu struktur data yang sering digunakan adalah Binary Tree. Struktur ini menyimpan data dalam bentuk hubungan antara parent dan child sehingga membentuk struktur hierarki. Selain itu, terdapat struktur data Graph yang digunakan untuk merepresentasikan hubungan antar objek yang lebih kompleks. Untuk menelusuri data pada graph, digunakan algoritma traversal seperti Depth First Search (DFS) dan Breadth First Search (BFS).

Oleh karena itu, pada praktikum ini dilakukan implementasi Binary Tree dan Graph Traversal menggunakan bahasa pemrograman Java agar mahasiswa dapat memahami cara kerja serta penerapan kedua struktur data tersebut.

1.2 Tujuan Praktikum

Adapun tujuan dari praktikum ini adalah:

1. Memahami konsep dan struktur Binary Tree sebagai salah satu struktur data non-linear.
2. Mempelajari cara membangun dan menghubungkan simpul (node) dalam sebuah Binary Tree.
3. Mengimplementasikan traversal Binary Tree menggunakan metode Inorder, Preorder, dan Postorder.
4. Memahami konsep Graph serta hubungan antar simpul dalam bentuk graf.

5. Mengimplementasikan algoritma Depth First Search (DFS) dan Breadth First Search (BFS) untuk melakukan penelusuran graph.
6. Menganalisis perbedaan hasil penelusuran yang dihasilkan oleh traversal Binary Tree, DFS, dan BFS.

1.3 Manfaat Praktikum

Adapun manfaat dari praktikum ini adalah:

1. Memberikan pemahaman mengenai cara penyimpanan data dalam bentuk hierarki menggunakan Binary Tree.
2. Membantu memahami proses penelusuran data pada struktur pohon melalui traversal Inorder, Preorder, dan Postorder.
3. Menambah wawasan mengenai representasi hubungan antar objek menggunakan struktur Graph.
4. Membantu memahami perbedaan cara kerja algoritma DFS dan BFS dalam menelusuri graph.

BAB II

PEMBAHASAN

2.1 Binary Tree dan Graph Traversal

Binary Tree adalah struktur data berbentuk pohon yang setiap simpulnya memiliki maksimal dua anak yaitu left child dan right child. Struktur ini digunakan untuk menyimpan data secara hierarkis sehingga mempermudah proses pencarian, traversal, dan pengolahan data. Pada praktikum ini dibangun sebuah Binary Tree yang terdiri dari sembilan simpul dengan nilai 1 sampai 9 yang saling terhubung membentuk struktur pohon.

Selain membangun struktur pohon, program juga mengimplementasikan tiga metode traversal, yaitu Inorder, Preorder, dan Postorder. Traversal Inorder mengunjungi subtree kiri terlebih dahulu, kemudian root, lalu subtree kanan. Traversal Preorder mengunjungi root terlebih dahulu sebelum menelusuri subtree kiri dan kanan. Sedangkan traversal Postorder mengunjungi subtree kiri dan kanan terlebih dahulu sebelum mengunjungi root. Ketiga traversal tersebut menghasilkan urutan data yang berbeda karena memiliki pola penelusuran yang berbeda pula.

Graph merupakan struktur data yang terdiri dari kumpulan simpul (vertex) dan hubungan antar simpul (edge). Pada praktikum ini digunakan graph tak berarah (undirected graph) yang terdiri dari simpul A, B, C, D, dan E. Untuk menelusuri graph digunakan algoritma DFS dan BFS.

DFS (Depth First Search) bekerja dengan menelusuri simpul sedalam mungkin sebelum kembali ke simpul sebelumnya. Sedangkan BFS (Breadth First Search) bekerja dengan mengunjungi seluruh tetangga simpul terlebih dahulu sebelum berpindah ke tingkat berikutnya. Kedua algoritma tersebut sangat berguna dalam pencarian jalur dan penelusuran graph.

2.2 Kode Pemograman

2.2.1 Class Node_2511531010

```
2 package pekan9_2511531010;
3
4 public class Node_2511531010 {
5     int data_1010;
6     Node_2511531010 left_1010;
7     Node_2511531010 right_1010;
8     public Node_2511531010(int data_1010) {
9         this.data_1010 = data_1010;
10        left_1010 = null;
11        right_1010 = null;
12    }
13    public void setLeft_1010(Node_2511531010 node_1010) {
14        if (left_1010 == null)
15            left_1010 = node_1010;
16    }
17    public void setRight_1010(Node_2511531010 node_1010) {
18        if (right_1010 == null)
19            right_1010 = node_1010;
20    }
21    public Node_2511531010 getLeft_1010() {
22        return left_1010;
23    }
24    public Node_2511531010 getRight_1010() {
25        return right_1010;
26    }
27    public int getData_1010() {
28        return data_1010;
29    }
30    public void setData_1010(int data_1010) {
31        this.data_1010 = data_1010;
32    }
33
34    void printPreorder_1010(Node_2511531010 node_1010) {
35        if (node_1010 == null)
36            return;
37        System.out.print(node_1010.data_1010 + " ");
38        printPreorder_1010(node_1010.left_1010);
39        printPreorder_1010(node_1010.right_1010);
40    }
41    void printPostorder_1010(Node_2511531010 node_1010) {
42        if (node_1010 == null)
43            return;
44        printPostorder_1010(node_1010.left_1010);
45        printPostorder_1010(node_1010.right_1010);
46        System.out.print(node_1010.data_1010 + " ");
47    }
48    void printInorder_1010(Node_2511531010 node_1010) {
49        if (node_1010 == null)
50            return;
51        printInorder_1010(node_1010.left_1010);
52        System.out.print(node_1010.data_1010 + " ");
```

```

53     printInorder_1010(node_1010.right_1010);
54 }
55 public String print_1010() {
56     return this.print_1010("", true, "");
57 }
58 public String print_1010(String prefix_1010, boolean
isTail_1010, String sb_1010) {
59     if (right_1010 != null) {
60         right_1010.print_1010(prefix_1010 + (isTail_1010 ? "| " :
"    "), false, sb_1010);
61     }
62     System.out.println(prefix_1010 + (isTail_1010 ? "\\-- " :
"/-- ") + data_1010);
63     if (left_1010 != null) {
64         left_1010.print_1010(prefix_1010 + (isTail_1010 ? "    "
: "| "), true, sb_1010);
65     }
66     return sb_1010;
67 }
68 }

```

Gambar 2.1

Class Node_2511531010 berfungsi sebagai representasi satu simpul pada Binary Tree. Setiap node menyimpan data bertipe integer serta memiliki referensi ke anak kiri (left_1010) dan anak kanan (right_1010). Class ini juga menyediakan method setter dan getter untuk mengakses data maupun hubungan antar node. Selain itu terdapat method traversal seperti printInorder_1010(), printPreorder_1010(), dan printPostorder_1010() yang digunakan untuk menampilkan isi pohon dengan urutan tertentu.

2.2.2 Class BTree_2511531010

```

2  package pekan9_2511531010;
3
4  public class BTree_2511531010 {
5      private Node_2511531010 root_1010;
6      private Node_2511531010 currentNode_1010;
7
8      public BTree_2511531010() {
9          root_1010 = null;
10     }
11     public boolean search_1010(int data_1010) {
12         return search_1010(root_1010, data_1010);
13     }
14     private boolean search_1010(Node_2511531010 node_1010, int
data_1010) {

```

```

15         if (node_1010.getData_1010() == data_1010)
16             return true;
17         if (node_1010.getLeft_1010() != null)
18             if (search_1010(node_1010.getLeft_1010(), data_1010))
19                 return true;
20         if (node_1010.getRight_1010() != null)
21             if (search_1010(node_1010.getRight_1010(), data_1010))
22                 return true;
23         return false;
24     }
25     public void printInorder_1010() {
26         root_1010.printInorder_1010(root_1010);
27     }
28     public void printPreOrder_1010() {
29         root_1010.printPreorder_1010(root_1010);
30     }
31     public void printPostOrder_1010() {
32         root_1010.printPostorder_1010(root_1010);
33     }
34     public Node_2511531010 getRoot_1010(){
35         return root_1010;
36     }
37     public boolean isEmpty_1010() {
38         return root_1010 == null;
39     }
40
41     public int countNodes_1010() {
42         return countNodes_1010(root_1010);
43     }
44
45     private int countNodes_1010(Node_2511531010 node_1010) {
46         int count_1010 = 1;
47         if (node_1010 == null) {
48             return 0;
49         } else {
50             count_1010 +=
51 countNodes_1010(node_1010.getLeft_1010());
52             count_1010 +=
53 countNodes_1010(node_1010.getRight_1010());
54             return count_1010;
55         }
56     }
57     public void print_1010() {
58         root_1010.print_1010();
59     }
60     public Node_2511531010 getCurrent_1010() {
61         return currentNode_1010;
62     }
63
64     public void setCurrent_1010(Node_2511531010 node_1010) {
65         this.currentNode_1010 = node_1010;
66     }

```

```

67
68     public void setRoot_1010(Node_2511531010 root_1010) {
69         this.root_1010 = root_1010;
70     }
71 }

```

Gambar 2.2

Class BTree_2511531010 digunakan untuk mengelola seluruh Binary Tree. Class ini memiliki root sebagai simpul utama pohon serta menyediakan berbagai operasi seperti pencarian data (search_1010()), menghitung jumlah simpul (countNodes_1010()), menampilkan traversal Inorder, Preorder, dan Postorder, serta menampilkan struktur pohon dalam bentuk visual. Class ini menjadi penghubung antara node-node yang telah dibuat dengan program utama.

2.2.3 Class BtreeDriver_2511531010

```

2  package pekan9_2511531010;
3
4  public class BtreeDriver_2511531010 {
5      public static void main (String[] args) {
6          // Membuat pohon
7          BTree_2511531010 tree_1010 = new BTree_2511531010 ();
8          System.out.print("Jumlah Simpul awal pohon: ");
9          System.out.println(tree_1010.countNodes_1010());
10         // menambahkan simpul data 1
11         Node_2511531010 root_1010 = new Node_2511531010 (1);
12         // menjadikan simpul 1 sebagai root
13         tree_1010.setRoot_1010(root_1010);
14         System.out.println("Jumlah simpul jika hanya ada root");
15         System.out.println(tree_1010.countNodes_1010());
16         Node_2511531010 node2_1010 = new Node_2511531010 (2);
17         Node_2511531010 node3_1010 = new Node_2511531010 (3);
18         Node_2511531010 node4_1010 = new Node_2511531010 (4);
19         Node_2511531010 node5_1010 = new Node_2511531010 (5);
20         Node_2511531010 node6_1010 = new Node_2511531010 (6);
21         Node_2511531010 node7_1010 = new Node_2511531010 (7);
22         Node_2511531010 node8_1010 = new Node_2511531010 (8);
23         Node_2511531010 node9_1010 = new Node_2511531010 (9);
24         root_1010.setLeft_1010(node2_1010);
25         node2_1010.setLeft_1010(node4_1010);
26         node2_1010.setRight_1010(node5_1010);
27         node4_1010.setRight_1010(node8_1010);
28         root_1010.setRight_1010(node3_1010);
29         node3_1010.setLeft_1010(node6_1010);
30         node3_1010.setRight_1010(node7_1010);

```

```

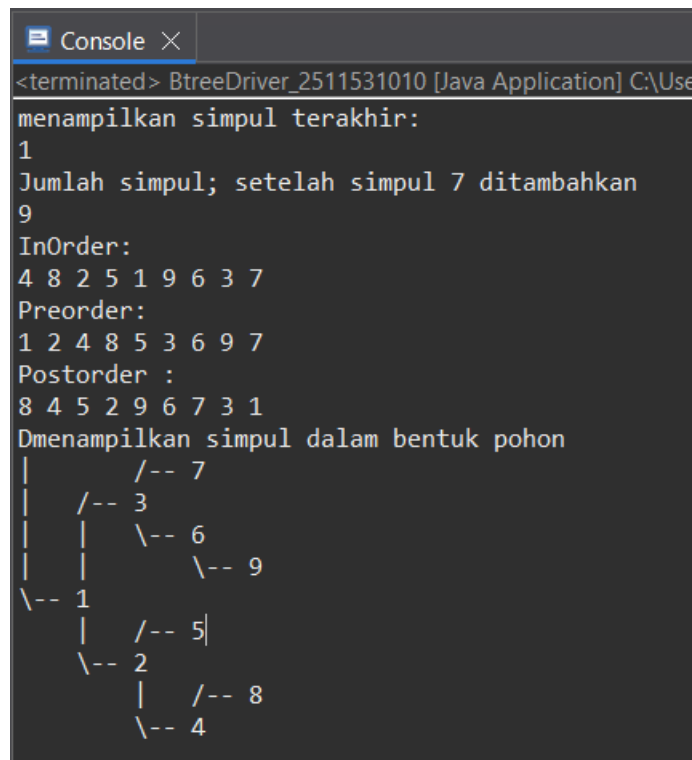
31     node6_1010.setLeft_1010(node9_1010);
32
33     // Set root
34     tree_1010.setCurrent_1010(tree_1010.getRoot_1010());
35     System.out.println("menampilkan simpul terakhir: ");
36
37     System.out.println(tree_1010.getCurrent_1010().getData_1010());
38     System.out.println("Jumlah simpul; setelah simpul 7
39     ditambahkan");
40     System.out.println(tree_1010.countNodes_1010());
41     System.out.println("InOrder: ");
42     tree_1010.printInorder_1010();
43     System.out.println("\nPreorder: ");
44     tree_1010.printPreOrder_1010();
45     System.out.println("\nPostorder : ");
46     tree_1010.printPostOrder_1010();
47     System.out.println("\nDmenampilkan simpul dalam bentuk
48     pohon");
49     tree_1010.print_1010();
50 }
51 }

```

Gambar 2.3

Class BtreeDriver_2511531010 merupakan program utama untuk Binary Tree. Pada class ini dibuat sembilan node dengan nilai 1 sampai 9, kemudian node-node tersebut dihubungkan menjadi sebuah struktur pohon biner. Setelah pohon terbentuk, program menampilkan jumlah simpul, nilai root, hasil traversal Inorder, Preorder, dan Postorder, serta menampilkan struktur pohon dalam bentuk visual.

2.2.4 Output BtreeDriver_2511531010



```
Console X
<terminated> BtreeDriver_2511531010 [Java Application] C:\Use
menampilkan simpul terakhir:
1
Jumlah simpul; setelah simpul 7 ditambahkan
9
InOrder:
4 8 2 5 1 9 6 3 7
Preorder:
1 2 4 8 5 3 6 9 7
Postorder :
8 4 5 2 9 6 7 3 1
Dmenampilkan simpul dalam bentuk pohon
|           /-- 7
|         /-- 3
|       |  \-- 6
|       |    \-- 9
|       |
|-- 1      /-- 5|
|         \-- 2
|           |  /-- 8
|           |  \-- 4
```

Gambar 2.4

Output pertama menampilkan jumlah simpul saat pohon masih kosong dan setelah root ditambahkan. Selanjutnya program menampilkan jumlah seluruh simpul setelah pohon selesai dibangun. Traversal Inorder menampilkan simpul dari kiri-root-kanan, traversal Preorder menampilkan root-kiri-kanan, sedangkan traversal Postorder menampilkan kiri-kanan-root. Pada bagian akhir program, struktur Binary Tree ditampilkan dalam bentuk diagram pohon sehingga hubungan antar simpul dapat terlihat dengan jelas.

2.2.5 Class GraphTraversal_2511531010

```
2 package pekan9_2511531010;
3
4 import java.util.*;
5 public class GraphTraversal_2511531010 {
6     private Map<String, List<String>> graph_1010 = new HashMap<>();
7
8     // Menambahkan edge (graf tak berarah)
9     public void addEdge(String node1_1010, String node2_1010) {
```

```

10     graph_1010.putIfAbsent(node1_1010, new ArrayList<>());
11     graph_1010.putIfAbsent(node2_1010, new ArrayList<>());
12     graph_1010.get(node1_1010).add(node2_1010);
13     graph_1010.get(node2_1010).add(node1_1010);
14 }
15
16 // Menampilkan graf awal
17 public void printGraph() {
18     System.out.println("Graf Awal (Adjacency List:");
19     for (String node_1010 : graph_1010.keySet()) {
20         System.out.print(node_1010 + " -> ");
21         List<String> neighbors_1010 =
graph_1010.get(node_1010);
22         System.out.println(String.join(", ", neighbors_1010));
23     }
24     System.out.println();
25 }
26
27 // DFS rekursif
28 public void dfs(String start) {
29     Set<String> visited_1010 = new HashSet<>();
30     System.out.println("Penelusuran DFS:");
31     dfsHelper(start, visited_1010);
32     System.out.println();
33 }
34
35 private void dfsHelper(String current_1010, Set<String>
visited_1010) {
36     if (visited_1010.contains(current_1010))
37         return;
38
39     visited_1010.add(current_1010);
40     System.out.print(current_1010 + " ");
41
42     for (String neighbor_1010 :
graph_1010.getDefault(current_1010, new ArrayList<>())) {
43         dfsHelper(neighbor_1010, visited_1010);
44     }
45 }
46
47 // BFS iteratif
48 public void bfs(String start) {
49     Set<String> visited_1010 = new HashSet<>();
50     Queue<String> queue_1010 = new LinkedList<>();
51
52     queue_1010.add(start);
53     visited_1010.add(start);
54
55     System.out.println("Penelusuran BFS:");
56
57     while (!queue_1010.isEmpty()) {
58         String current_1010 = queue_1010.poll();
59         System.out.print(current_1010 + " ");
60

```

```

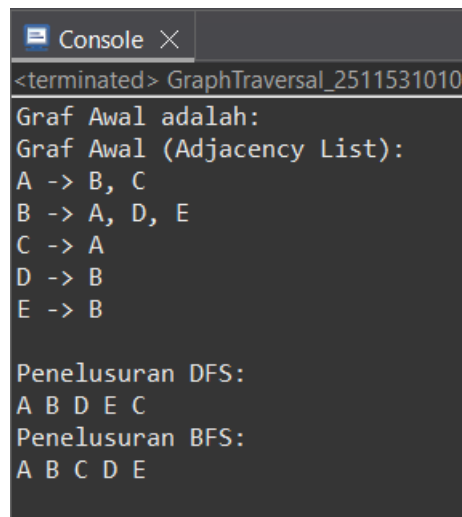
61         for (String neighbor_1010 :
graph_1010.getDefault(current_1010, new ArrayList<>())) {
62             if (!visited_1010.contains(neighbor_1010)) {
63                 queue_1010.add(neighbor_1010);
64                 visited_1010.add(neighbor_1010);
65             }
66         }
67     }
68     System.out.println();
69 }
70
71 // Main
72 public static void main(String[] args) {
73     GraphTraversal_2511531010 graph_1010 = new
GraphTraversal_2511531010();
74
75     // Contoh graf: A-B, A-C, B-D, B-E
76     graph_1010.addEdge("A", "B");
77     graph_1010.addEdge("A", "C");
78     graph_1010.addEdge("B", "D");
79     graph_1010.addEdge("B", "E");
80
81     // Cetak graf awal
82     System.out.println("Graf Awal adalah: ");
83     graph_1010.printGraph();
84
85     // Lakukan penelusuran
86     graph_1010.dfs("A");
87     graph_1010.bfs("A");
88 }
89 }

```

Gambar 2.5

Class GraphTraversal_2511531010 digunakan untuk membuat dan menelusuri graph. Program menyimpan graph menggunakan struktur HashMap yang berisi daftar tetangga setiap simpul. Method addEdge() digunakan untuk menambahkan hubungan antar simpul. Selain itu terdapat implementasi algoritma DFS menggunakan rekursi dan BFS menggunakan Queue. Program juga menyediakan method untuk menampilkan adjacency list dari graph yang telah dibuat.

2.2.6 Output GraphTraversal_2511531010



```
<terminated> GraphTraversal_2511531010
Graf Awal adalah:
Graf Awal (Adjacency List):
A -> B, C
B -> A, D, E
C -> A
D -> B
E -> B

Penelusuran DFS:
A B D E C
Penelusuran BFS:
A B C D E
```

Gambar 2.6

Output pertama menampilkan graph dalam bentuk adjacency list yang menunjukkan hubungan antar simpul. Setelah itu program menampilkan hasil traversal DFS dimulai dari simpul A, di mana penelusuran dilakukan sedalam mungkin sebelum kembali ke simpul sebelumnya. Selanjutnya program menampilkan hasil traversal BFS yang menelusuri simpul berdasarkan tingkat kedekatannya dari simpul awal. Perbedaan urutan hasil DFS dan BFS menunjukkan perbedaan cara kerja kedua algoritma tersebut.

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Binary Tree dan Graph merupakan struktur data non-linear yang memiliki peran penting dalam penyimpanan serta pengolahan data yang saling berhubungan. Melalui implementasi Binary Tree, dapat dipahami bagaimana data disusun dalam bentuk hierarki menggunakan node yang saling terhubung serta bagaimana proses traversal dilakukan melalui metode Inorder, Preorder, dan Postorder untuk menghasilkan urutan penelusuran yang berbeda.

Selain itu, implementasi Graph Traversal memberikan pemahaman mengenai cara menelusuri hubungan antar simpul menggunakan algoritma Depth First Search (DFS) dan Breadth First Search (BFS). Kedua algoritma tersebut memiliki cara kerja yang berbeda, di mana DFS menelusuri simpul hingga kedalaman tertentu terlebih dahulu, sedangkan BFS menelusuri simpul berdasarkan tingkat kedekatannya dari simpul awal.

DAFTAR PUSTAKA

- [1] Oracle Corporation. 2023. *Java Documentation*. Diakses dari <https://docs.oracle.com/javase/8/docs/>

TUGAS PRAKTIKUM

Soal:

Deskripsi Tugas

Mahasiswa diminta untuk mengimplementasikan algoritma BFS (*Breadth First Search*) dan DFS (*Depth First Search*) menggunakan bahasa Java dengan antarmuka GUI.

Studi kasus yang digunakan adalah pencarian jalur pada suatu peta lokasi yang direpresentasikan dalam bentuk Graph.

Setiap mahasiswa wajib membuat peta lokasi yang berbeda dengan mahasiswa lain. Peta dapat berupa:

- Gedung Perkuliahan
- Fakultas
- Universitas
- Sekolah
- Rumah Sakit
- Mall
- Tempat Wisata
- Bandara
- Atau lokasi lainnya

Program harus mampu:

- Menampilkan graph dalam bentuk visual
- Menentukan titik awal (Start)
- Menentukan titik tujuan (Goal)
- Melakukan pencarian menggunakan BFS
- Melakukan pencarian menggunakan DFS
- Menampilkan jalur yang ditemukan

- Menampilkan urutan simpul yang dikunjungi
- Urutan node yang dikunjungi
- Jumlah node yang dieksplorasi

Aturan Penamaan

1. Penamaan class dan file

Gunakan akhiran NIM lengkap pada nama class dan nama file.

Contoh:

ex : PetaKampus_2311532008

2. Penamaan Variabel

Gunakan 4 digit terakhir pada setiap nama variabel

ex : graph_2019

Struktur Tugas

Program minimal memiliki:

a. Vertex and Edge

- Minimal 10 simpul (vertex)
- Minimal 15 sisi (edge)
- Nama simpul harus merepresentasikan lokasi pada peta yang dibuat

b. Method yang Wajib Ada

1. BFS()

Melakukan pencarian menggunakan algoritma Breadth First Search.

2. DFS()

Melakukan pencarian menggunakan algoritma Depth First Search.

3. displayGraph()

Menampilkan graph pada GUI.

4. displayPath()

Menampilkan jalur yang ditemukan.

5. `resetGraph()`

Mengembalikan kondisi graph ke keadaan awal.

c. GUI Program

GUI minimal memiliki:

1. Area visualisasi graph
2. ComboBox atau TextField untuk memilih lokasi awal
3. ComboBox atau TextField untuk memilih lokasi tujuan
4. Tombol BFS
5. Tombol DFS
6. Tombol Reset
7. Area hasil pencarian
8. berikan warna berbeda pada node yang telah dikunjungi.
9. Menampilkan jumlah node yang dieksplorasi.

d. Ketentuan Graph

1. Setiap mahasiswa wajib membuat graph yang berbeda.
2. Tidak diperbolehkan menggunakan graph yang sama persis dengan mahasiswa lain.
3. Graph harus memiliki minimal 10 node dan 15 edge.
4. Mahasiswa bebas menentukan bentuk dan jenis peta.
5. Mahasiswa wajib menjelaskan struktur graph, cara kerja BFS dan DFS yang dibuat pada laporan.

Kode Pemograman:

```
package pekan9_2511531010;

import javax.swing.*;
import java.awt.*;
import java.util.*;

public class RumahSakit_2511531010 extends JFrame {

    private JComboBox<String> cmbStart_1010, cmbGoal_1010;
    private JTextArea txtHasil_1010;
    private GraphPanel_1010 graphPanel_1010;

    private Map<String, java.util.List<String>> graph_1010 = new
    LinkedHashMap<>();
    private Set<String> visited_1010 = new LinkedHashSet<>();
    private String mode_1010 = "";

    public void displayGraph_1010() {
```

```

        graphPanel_1010.repaint();
    }

    public void displayPath_1010(String hasil_1010) {
        txtHasil_1010.setText(hasil_1010);
    }

    public RumahSakit_2511531010() {
        initGraph_1010();

        setTitle("PENCARIAN JALUR MENGGUNAKAN BFS DAN DFS");
        setSize(1200, 850);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLayout(null);

        JLabel title = new JLabel("PENCARIAN JALUR MENGGUNAKAN BFS DAN
DFS", SwingConstants.CENTER);
        title.setBounds(0, 0, 1200, 60);
        title.setOpaque(true);
        title.setBackground(new Color(25,60,120));
        title.setForeground(Color.WHITE);
        title.setFont(new Font("Arial", Font.BOLD, 24));
        add(title);

        add(new JLabel("Lokasi Awal :")).setBounds(30,80,100,25);
        cmbStart_1010 = new JComboBox<>(graph_1010.keySet().toArray(new
String[0]));
        cmbStart_1010.setBounds(140,80,220,25);
        add(cmbStart_1010);

        add(new JLabel("Lokasi Tujuan :")).setBounds(30,120,100,25);
        cmbGoal_1010 = new JComboBox<>(graph_1010.keySet().toArray(new
String[0]));
        cmbGoal_1010.setBounds(140,120,220,25);
        add(cmbGoal_1010);

        JButton bfs = new JButton("BFS");
        bfs.setBounds(500,80,100,35);
        add(bfs);

        JButton dfs = new JButton("DFS");
        dfs.setBounds(620,80,100,35);
        add(dfs);

        JButton reset = new JButton("RESET");
        reset.setBounds(740,80,100,35);
        add(reset);

        graphPanel_1010 = new GraphPanel_1010();

        graphPanel_1010.setBorder(BorderFactory.createTitledBorder("VISUALISASI
GRAPH"));
        graphPanel_1010.setBounds(30,170,1100,380);
    }

```

```

add(graphPanel_1010);

txtHasil_1010 = new JTextArea();
txtHasil_1010.setEditable(false);
JScrollPane sp = new JScrollPane(txtHasil_1010);
sp.setBounds(30,580,1100,180);
add(sp);

bfs.addActionListener(e -> BFS_1010());
dfs.addActionListener(e -> DFS_1010());
reset.addActionListener(e -> resetGraph_1010());
}

private void initGraph_1010(){
    addEdge("Parkiran","Administrasi");
    addEdge("Administrasi","Laboratorium");
    addEdge("Laboratorium","Radiologi");
    addEdge("Radiologi","Ruang Operasi");
    addEdge("Ruang Operasi","ICU");
    addEdge("ICU","Rawat Inap");
    addEdge("IGD","Rawat Inap");
    addEdge("IGD","Radiologi");
    addEdge("IGD","Rawat Jalan");
    addEdge("Rawat Jalan","Apotek");
    addEdge("Apotek","Laboratorium");
    addEdge("Rawat Jalan","Parkiran");
    addEdge("Administrasi","Apotek");
    addEdge("Rawat Inap","Radiologi");
    addEdge("ICU","Radiologi");
}

private void addEdge(String a,String b){
    graph_1010.putIfAbsent(a,new ArrayList<>());
    graph_1010.putIfAbsent(b,new ArrayList<>());
    graph_1010.get(a).add(b);
    graph_1010.get(b).add(a);
}

public void BFS_1010(){ search(true); }
public void DFS_1010(){ search(false); }

private void search(boolean bfs){
    String start=(String)cmbStart_1010.getSelectedItem();
    String goal=(String)cmbGoal_1010.getSelectedItem();

    Map<String,String> parent=new LinkedHashMap<>();
    visited_1010.clear();

    if(bfs){
        mode_1010="BFS";
        Queue<String> q=new LinkedList<>();
        q.add(start); visited_1010.add(start);
        while(!q.isEmpty()){
            String c=q.poll();

```

```

        if(c.equals(goal)) break;
        for(String n:graph_1010.get(c))
            if(!visited_1010.contains(n)){
                visited_1010.add(n); parent.put(n,c); q.add(n);
            }
    }
    }else{
        mode_1010="DFS";
        Stack<String> s=new Stack<>();
        s.push(start);
        while(!s.isEmpty()){
            String c=s.pop();
            if(!visited_1010.contains(c)){
                visited_1010.add(c);
                if(c.equals(goal)) break;
                for(String n:graph_1010.get(c))
                    if(!visited_1010.contains(n)){
                        parent.put(n,c); s.push(n);
                    }
            }
        }
    }

    ArrayList<String> path=new ArrayList<>();
    String cur=goal;
    while(cur!=null){ path.add(cur); cur=parent.get(cur); }
    Collections.reverse(path);

    displayPath_1010(
        "Hasil Pencarian : " + mode_1010 +
        "\n\nJalur : " + path +
        "\n\nNode Dikunjungi : " + visited_1010 +
        "\n\nJumlah Node Dikunjungi : " + visited_1010.size()
    );

    displayGraph_1010();
}

public void resetGraph_1010(){
    visited_1010.clear();
    mode_1010="";
    txtHasil_1010.setText("");
    graphPanel_1010.repaint();
}

class GraphPanel_1010 extends JPanel{
    Map<String,Point> p=new HashMap<>();
    GraphPanel_1010(){
        p.put("Parkiran",new Point(150,60));
        p.put("Administrasi",new Point(450,60));
        p.put("Rawat Jalan",new Point(120,150));
        p.put("Apotek",new Point(300,150));
        p.put("Laboratorium",new Point(500,150));
        p.put("IGD",new Point(180,260));
    }
}

```

```

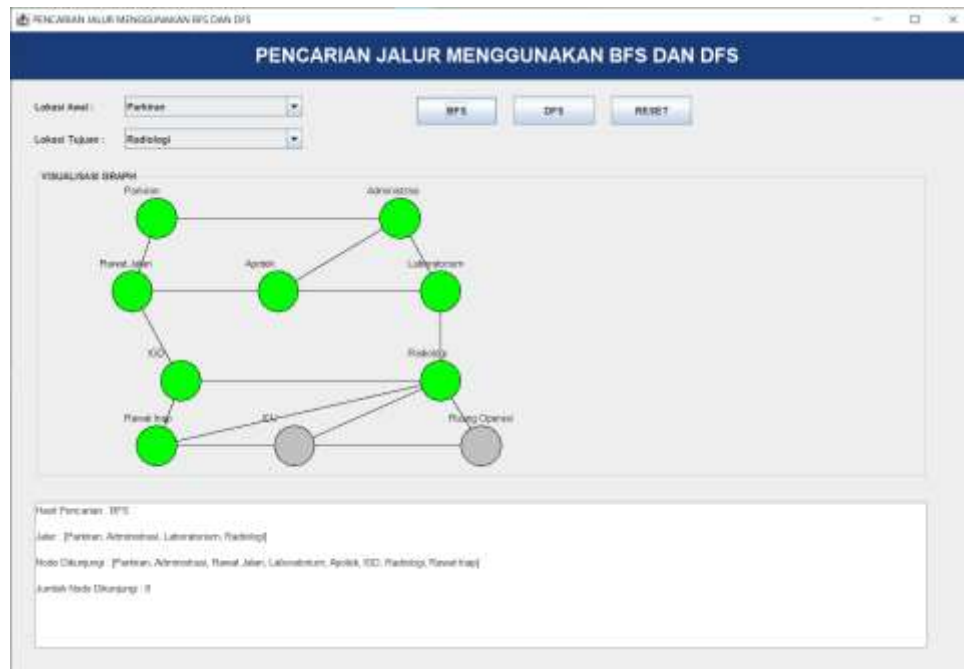
        p.put("Radiologi", new Point(500, 260));
        p.put("Rawat Inap", new Point(150, 340));
        p.put("ICU", new Point(320, 340));
        p.put("Ruang Operasi", new Point(550, 340));
    }
    protected void paintComponent(Graphics g){
        super.paintComponent(g);
        g.setColor(Color.BLACK);
        for(String a:graph_1010.keySet())
            for(String b:graph_1010.get(a)){
                Point p1=p.get(a), p2=p.get(b);
                g.drawLine(p1.x,p1.y,p2.x,p2.y);
            }
        for(String n:p.keySet()){
            Point pt=p.get(n);
            if(visited_1010.contains(n)){
                g.setColor(mode_1010.equals("DFS"?Color.ORANGE:Color.GREEN);
            } else g.setColor(Color.LIGHT_GRAY);
            g.fillOval(pt.x-25,pt.y-25,50,50);
            g.setColor(Color.BLACK);
            g.drawOval(pt.x-25,pt.y-25,50,50);
            g.drawString(n, pt.x-40, pt.y-30);
        }
    }
}

    public static void main(String[] args){
        SwingUtilities.invokeLater(() -> new
RumahSakit_2511531010().setVisible(true));
    }
}

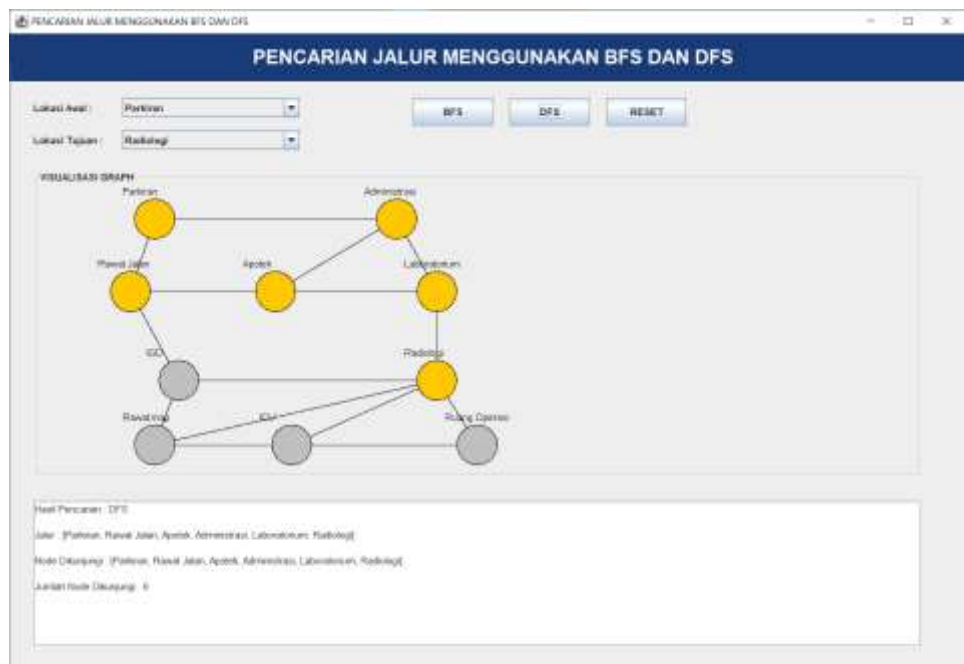
```

Screenshot program saat dijalankan.

1. BFS



2. DFS



Deskripsi Peta yang Dibuat

Pada tugas ini dibuat sebuah graph yang merepresentasikan peta lokasi pada sebuah rumah sakit. Setiap simpul (vertex) mewakili ruangan atau fasilitas yang ada di rumah sakit, sedangkan sisi (edge) menunjukkan hubungan atau jalur yang dapat dilalui antar lokasi.

Lokasi yang digunakan pada graph adalah:

1. Parkiran
2. Administrasi
3. Rawat Jalan
4. Apotek
5. Laboratorium
6. IGD
7. Radiologi
8. Rawat Inap
9. ICU
10. Ruang Operasi

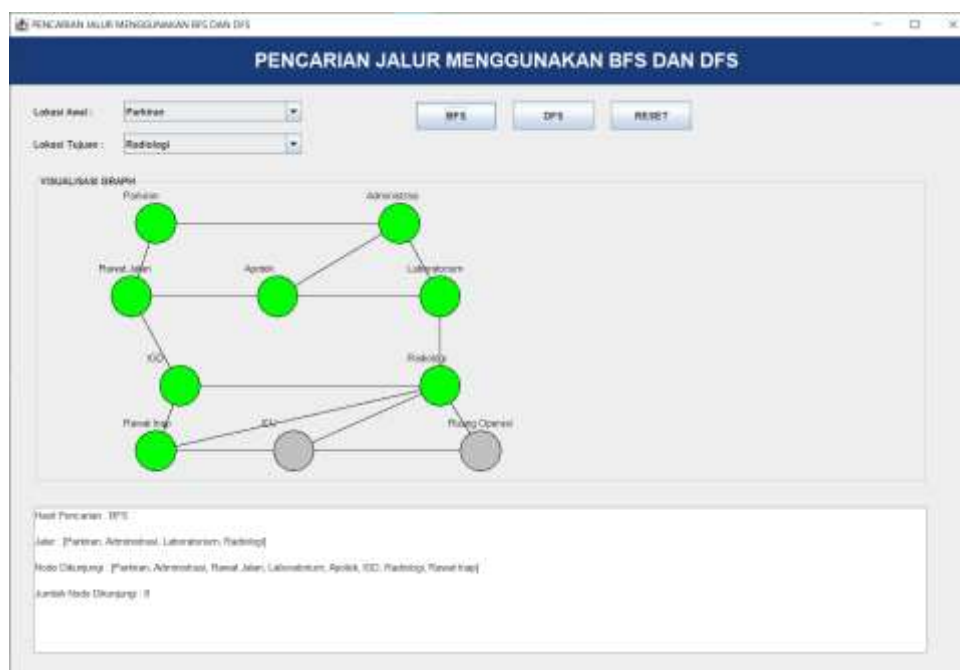
Jumlah Node (Vertex):

1. Parkiran
2. Administrasi
3. Rawat Jalan
4. Apotek
5. Laboratorium
6. IGD
7. Radiologi
8. Rawat Inap
9. ICU
10. Ruang Operasi

Jumlah Edge

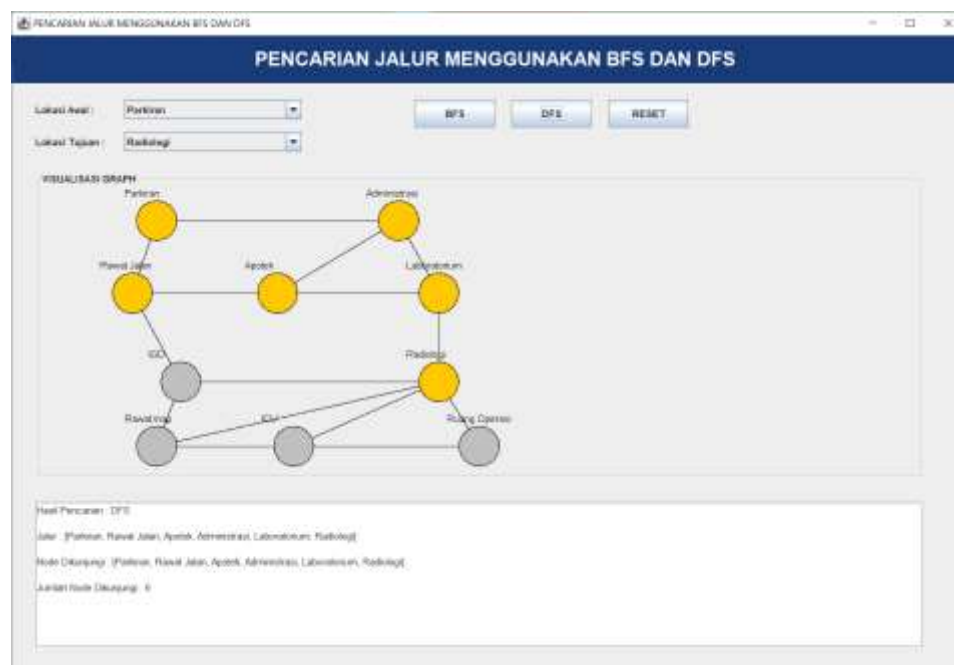
1. Parkiran – Administrasi
2. Administrasi – Laboratorium
3. Laboratorium – Radiologi
4. Radiologi – Ruang Operasi
5. Ruang Operasi – ICU
6. ICU – Rawat Inap
7. IGD – Rawat Inap
8. IGD – Radiologi
9. IGD – Rawat Jalan
10. Rawat Jalan – Apotek
11. Apotek – Laboratorium
12. Rawat Jalan – Parkiran
13. Administrasi – Apotek
14. Rawat Inap – Radiologi
15. ICU – Radiologi

Hasil BFS



Algoritma BFS melakukan pencarian secara melebar (level per level). BFS akan mengunjungi semua node yang berada pada tingkat terdekat terlebih dahulu sebelum melanjutkan ke tingkat berikutnya. Karena itu BFS mampu menemukan jalur terpendek dari Parkiran menuju Radiologi.

Hasil DFS



Algoritma DFS melakukan pencarian dengan menelusuri satu cabang sedalam mungkin terlebih dahulu sebelum kembali ke cabang sebelumnya. Pada kasus ini DFS menemukan Radiologi setelah melewati jalur yang lebih panjang dibandingkan BFS.

Kesimpulan Perbandingan BFS dan DFS

Berdasarkan hasil pengujian pada graph rumah sakit, algoritma BFS dan DFS sama-sama berhasil menemukan jalur dari Parkiran menuju Radiologi. Namun, kedua algoritma memiliki cara kerja yang berbeda.

BFS mencari node secara melebar sehingga dapat menemukan jalur terpendek. Pada percobaan ini BFS menghasilkan jalur:

Parkiran → Administrasi → Laboratorium → Radiologi

Sedangkan DFS mencari node secara mendalam terlebih dahulu sehingga jalur yang ditemukan bergantung pada urutan traversal node. Pada percobaan ini DFS menghasilkan jalur:

Parkiran → Rawat Jalan → Apotek → Administrasi → Laboratorium → Radiologi

Dapat disimpulkan bahwa BFS lebih cocok digunakan untuk mencari jalur terpendek, sedangkan DFS lebih cocok digunakan untuk penelusuran graph secara mendalam dan eksplorasi struktur graph. Pada kasus ini BFS memberikan hasil yang lebih optimal karena menemukan jalur yang lebih singkat menuju tujuan.